

4. Methodology

We use a simulation-based robust class-selection framework to determine the structural class from which a wave is released. Given the candidate pool and class definitions in Section 3, candidate waves are evaluated under the two deterministic elevator models using common order data and prescribed reservation rules. Each class is scored by the larger of its two median makespans. The framework selects the class with the smallest such score and implements the decision by drawing a candidate wave from its fixed within-class distribution.

Two analyses support this selection framework. A class-level diagnostic measures performance differences across the selectable classes and the loss associated with a prespecified class-selection rule. A paired analysis of elevator requests establishes sufficient conditions for ordering the two evaluators and identifies mechanisms that can reverse that ordering. When the M_2 median is no smaller in every selectable class, the robust decision reduces to selecting the class with the smallest M_2 median; departures from this ordering are assessed through class-score differences and ranking margins. Fig. 2 shows the decision flow and the supporting roles of the three analyses. Section 4.5 uses these results to build a release procedure that works with individual candidate waves, and Section 4.6 states what Section 5 tests.

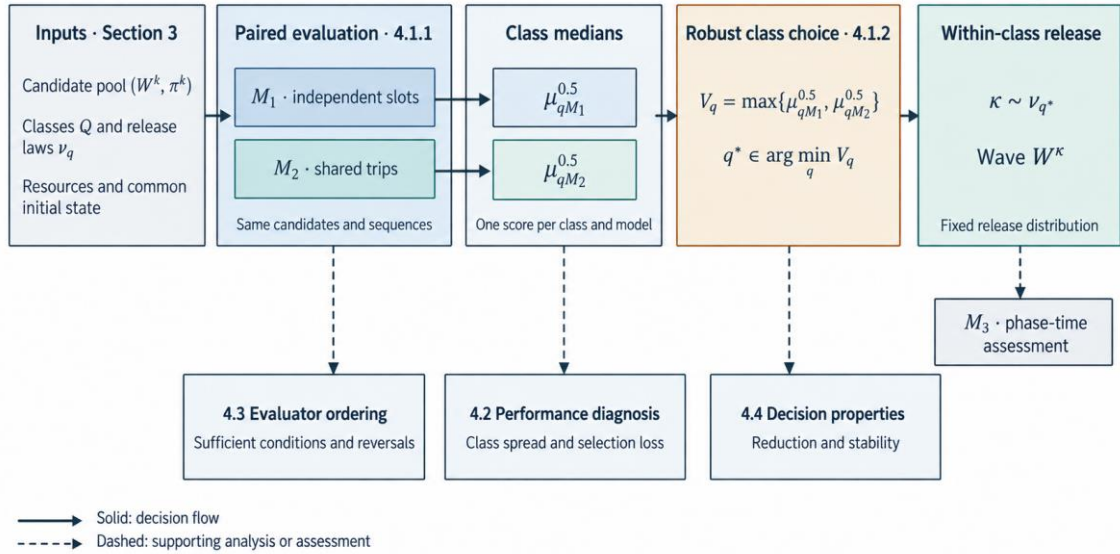


Fig. 2. Robust class selection and its supporting analyses. Common recorded order sequences support paired evaluations under M_1 and M_2 . Evaluator-specific class medians enter the minimax rule, and the selected class determines the fixed distribution from which a wave is released. The diagram displays exact scores; sampled scores follow the same flow with the estimates in Equations (34)–(35). Class-relative diagnosis, conditional evaluator ordering, and decision-property analysis form supporting branches. M_3 assesses the selected class under phase-time variation. Solid arrows indicate the decision flow; dashed arrows indicate supporting analysis or assessment.

4.1 Robust class-selection procedure

4.1.1 Candidate evaluation and class scores

The procedure takes the candidate pool, structural classes, resource configuration, and within-class release distributions defined in Section 3 as inputs. Each candidate is evaluated under M_1 and M_2 using the same order sequence and initial state. For each class and evaluator, the deterministic median is obtained exactly from one equally weighted outcome per candidate in its support, retaining repeated outcome values:

$$\mu_{qm}^{0.5} = \text{Med}_{k \in \mathcal{K}_q} Y_{km}, \quad m \in \mathcal{M}_D \quad (33)$$

When class scores are estimated by sampling, R_q candidate indices $\kappa_{qr} \sim \nu_q$ are drawn independently, and the same draws are used across evaluators. For evaluations under M_3 , phase-time realizations ξ_{qr} are independent across replications and independent of the candidate indices. The estimated class median is

$$\hat{\mu}_{qm}^{0.5} = \text{Med}_{r=1, \dots, R_q} Y_{\kappa_{qr}m}(\xi_{qr}) \quad (34)$$

where ξ_{qr} is omitted for deterministic evaluators. For M_1 and M_2 , candidate sampling is the source of variation in this estimate. Under M_3 , it also incorporates phase-time variation, matching the joint outcome distribution in Equation (14). All medians follow the midpoint convention in Section 3.4.2. Section 5 specifies the sampling sizes and uncertainty analysis.

4.1.2 Class selection and wave release

For each class, the decision rule retains the larger median across $\mathcal{M}_D = \{M_1, M_2\}$ and selects the class with the smallest retained value. Thus, the comparison is between evaluator-specific class medians. With exact scores, the selected class is q^* from Equation (15). With estimated scores, the same rule gives

$$\hat{q} \in \arg \min_{q \in \mathcal{Q}} \max_{m \in \mathcal{M}_D} \hat{\mu}_{qm}^{0.5} \quad (35)$$

A fixed class-label order resolves ties in either minimization. The selected class is implemented through Equation (12), with $\kappa \sim \nu_{q^*}$ for the exact-score decision and $\kappa \sim \nu_{\hat{q}}$ for the estimated-score decision. The resulting wave W^* is then released. This step maps the class decision to a concrete wave while preserving the specified within-class sampling rule. The resulting output comprises the selected class, its release distribution, and its evaluated makespan profile. The direct candidate benchmark in Equations (16)–(17) provides a comparison with choosing an individual wave, while M_3 assesses performance under phase-time variation. The same class-score data also support the performance diagnosis developed next.

4.2 Class-relative performance diagnosis

4.2.1 Reference scores and diagnostic decomposition

Alongside the robust decision, the evaluation outcomes quantify the performance differences between classes and the quality of a prespecified selection rule. For a fixed resource configuration, wave size, and evaluator m , the class medians in Equation (14) provide the comparison scores. The

reference choice draws uniformly from the entire candidate pool. This pool distribution is denoted by ν_0 , while b_0 and b_q denote the reference and class medians under the fixed evaluator:

$$\nu_0(k) = \frac{1}{K}, \quad b_0 = \text{Med}_{\kappa \sim \nu_0, \xi} Y_{\kappa m}(\xi), \quad b_q = \mu_{qm}^{0.5}, \quad q \in \mathcal{Q} \quad (36)$$

For M_1 and M_2 , the pool reference is computed by complete candidate enumeration or estimated using draws from ν_0 . Under M_3 , it is estimated from joint candidate and phase-time draws following Section 4.1. All medians use the midpoint convention in Section 3.4.2. Positive service durations and a nonempty wave give $b_0 > 0$, so it can normalize comparisons across configurations. The best and worst selectable classes under this evaluator are denoted by

$$q_{\min} \in \arg \min_{q \in \mathcal{Q}} b_q, \quad q_{\max} \in \arg \max_{q \in \mathcal{Q}} b_q, \quad b_- = b_{q_{\min}}, \quad b_+ = b_{q_{\max}} \quad (37)$$

The class returned by a prespecified selection rule using the descriptors Φ is denoted by $q_\Phi \in \mathcal{Q}$. Its fitting procedure, treatment of tied scores, and handling of empty corner supports form part of that rule. The diagnostic takes this rule's selected class as an input and compares it with both the class oracle $q_{\min}$ and the pool reference. Here q_Φ identifies the class selected by the rule being assessed, whereas q^* and $\hat{q}$ identify the robust decisions defined above. The normalized class spread UB , the selected class's normalized gain LB , and their difference GAP are defined as

$$\text{UB} = \frac{b_+ - b_-}{b_0}, \quad \text{LB} = \frac{b_0 - b_{q_\Phi}}{b_0}, \quad \text{GAP} = \text{UB} - \text{LB} \quad (38)$$

Proposition 1 (diagnostic identity). For any nonempty selectable class family and any selected class q_Φ , the diagnostic satisfies

$$\text{GAP} = H_{\text{up}} + M_\Phi, \quad H_{\text{up}} = \frac{b_+ - b_0}{b_0}, \quad M_\Phi = \frac{b_{q_\Phi} - b_-}{b_0} \quad (39)$$

Proof. Substituting Equation (38) gives $b_+ - b_- - b_0 + b_{q_\Phi} = (b_+ - b_0) + (b_{q_\Phi} - b_-)$ in the numerator. Division by b_0 yields Equation (39). The two terms describe different comparisons. The class-family-relative upper-tail headroom H_{up} locates the largest class median relative to the pool median. The class-selection miss M_Φ measures the normalized excess median makespan of the selected class over the best class in the same family. Resource quantities and operational rules remain fixed in both comparisons. Because $b_- \leq b_{q_\Phi} \leq b_+$,

$$\text{UB} \geq 0, \quad 0 \leq M_\Phi \leq \text{UB} \quad (40)$$

The signs of the other terms depend on the reference comparison: $H_{\text{up}} \geq 0$ exactly when $b_+ \geq b_0$, and $\text{LB} \geq 0$ exactly when the selected class's median is no greater than the pool median. The corner family in Section 3.3 need not cover the entire candidate pool and may have overlapping supports at tied thresholds. Its class medians therefore need not bracket b_0 ; H_{up} and GAP can be negative. These values retain the signed comparisons defined in Equations (38)–(39).

4.2.2 Class-selection loss

The selection term also has a decision-loss interpretation. In predict-then-optimize, the Smart Predict-then-Optimize (SPO) loss evaluates the cost of the decision induced by predicted coefficients, relative to an optimal decision under the true coefficients (Adam N. Elmachetoub & Paul Grigas, 2021).

Here the actions are class labels. Their cost vector is $\mathbf{d} = (b_q)_{q \in \mathcal{Q}}$, and a predicted class-score vector

$\hat{\mathbf{d}}$ induces the choice $q_\Phi \in \arg \min_q \hat{d}_q$ under a fixed tie convention.

Remark 1 (class-action loss interpretation). For the class decision induced by $\hat{\mathbf{d}}$, its SPO loss under $\mathbf{d}$ satisfies

$$L_{\text{SPO}}(\mathbf{d}, \hat{\mathbf{d}}) = b_{q_\Phi} - \min_{q \in \mathcal{Q}} b_q = b_0 M_\Phi \quad (41)$$

Proof. The selected action incurs cost b_{q_Φ} , and the oracle action incurs cost b_- . Their difference is the numerator of M_Φ . Thus, M_Φ evaluates the quality of a class decision on a fixed class-cost vector. A learning method can target this loss through the class scores it predicts; its training and generalization properties depend on the learning procedure and data design.

4.2.3 Resolution of covering partitions

The resolution analysis uses a finite, disjoint, covering partition $\mathcal{P}$ of the same candidate pool to examine how classification detail changes the observed class spread. Each cell has positive probability under ν_0 , and its outcome distribution is obtained by conditioning the same pool distribution and evaluator on that cell. Its midpoint median is denoted by b_p . This construction preserves the mixture relationship between the pool and its cells, giving

$$\min_{p \in \mathcal{P}} b_p \leq b_0 \leq \max_{p \in \mathcal{P}} b_p \quad (42)$$

The bracketing property holds for the midpoint convention. In the proof of the upper inequality, $t = \max_p b_p$, and $[u_p, v_p]$ denotes the median interval of cell p . Every cell has cumulative probability at least one half at t . If the pool midpoint exceeded t , the pool's cumulative probability at t would equal one half, forcing the same equality in every positive-weight cell. Their median intervals would then have a common point, and the pool median interval would be their intersection. A cell attaining the largest lower endpoint would have a midpoint at least as large as the pool midpoint, contradicting the definition of t . Reflection of the outcome variable gives the lower inequality.

Proposition 2 (nested-partition refinement). The following result concerns finite covering partitions $\mathcal{P}$ and $\mathcal{P}'$ of the same candidate pool, with positive-probability cells and cell distributions induced by the same base distribution and evaluator. If $\mathcal{P}'$ refines $\mathcal{P}$, meaning that every cell of $\mathcal{P}'$ lies within one cell of $\mathcal{P}$, then

$$\max_{p' \in \mathcal{P}'} b_{p'} \geq \max_{p \in \mathcal{P}} b_p, \quad \min_{p' \in \mathcal{P}'} b_{p'} \leq \min_{p \in \mathcal{P}} b_p \quad (43)$$

Proof. The outcome distribution in a parent cell is the mixture of the distributions in its children, with

weights $v_0(p')/v_0(p)$. Equation (42), applied within that parent, brackets its median by its smallest and largest child medians. The upper inequality for a parent attaining the largest coarse median and the lower inequality for a parent attaining the smallest coarse median establish the result.

Diagnostic resolution. Along a nested sequence of such partitions, $H_{\text{up}}(\mathcal{P})$, the oracle gain

$S_{\text{or}}(\mathcal{P}) = [b_0 - \min_p b_p] / b_0$, and $\text{UB}(\mathcal{P}) = H_{\text{up}}(\mathcal{P}) + S_{\text{or}}(\mathcal{P})$ are nonnegative and nondecreasing under

refinement. This follows from Equations (42)–(43), with b_0 fixed. The selection term M_ϕ additionally depends on the rule's choice at each resolution. The corner family $\mathcal{Q}$ and the covering partition $\mathcal{P}$ answer different diagnostic questions. The former compares selected tail combinations of $\mathcal{C}$ and $\mathcal{I}$; the latter tracks how a complete classification resolves variation across the whole pool. Resolution comparisons use genuinely nested boundaries and consistent allocation of threshold ties. At the sample level, exact checks of Proposition 2 partition the same weighted outcome sample so that each parent distribution remains the mixture of its children. Fig. 3 separates the class-relative comparisons from the resolution analysis of covering partitions.

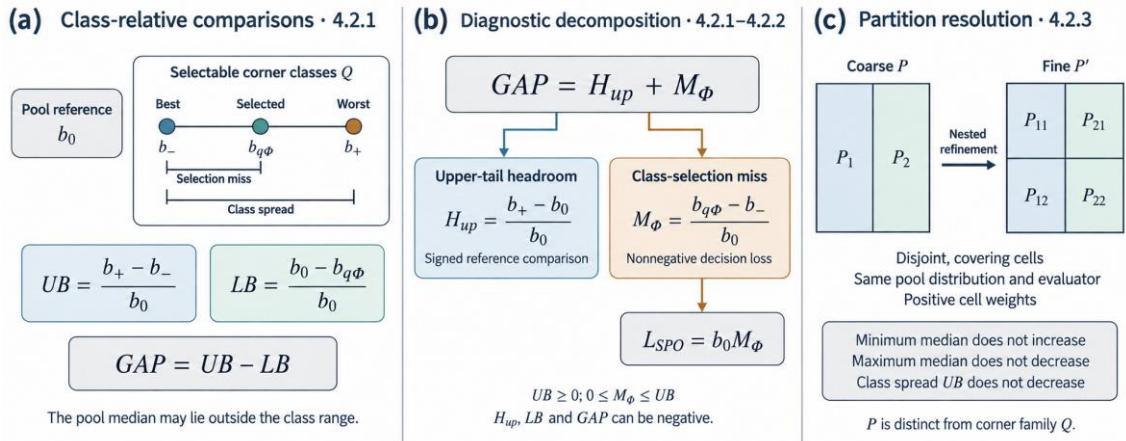


Fig. 3. Comparisons underlying the class-relative diagnostic. (a) Under one fixed evaluator, the best, selected, and worst class medians define the class spread and selected gain relative to the pool median b_0 . (b) The identity $\text{GAP} = H_{\text{up}} + M_\phi$ separates the signed reference comparison from the nonnegative class-selection miss; $b_0 M_\phi$ is the corresponding class-action loss. (c) A separate nested, disjoint, covering partition uses the same pool distribution and evaluator, with positive cell weights. Refinement cannot shrink the range of cell medians. This partition is distinct from the corner family $\mathcal{Q}$.

4.3 Conditional ordering of the deterministic evaluators

4.3.1 Coupled request comparison

The relative magnitudes of the two evaluator scores determine which model sets a class's robust score. To analyze this relationship, we compare their resource reservations. The deterministic evaluators represent evaluator capacity through different resource states: M_1 uses E_c independent single-request slots, whereas M_2 uses E physical cars with shared trips. Their completion times depend jointly on request timing, resource availability, and stored destination floors. We compare the evaluators on the same candidate wave using the serial reservation rules of Section 3.5. A

corresponding elevator request is denoted by ϱ , and its unloading completion time under M_i by $D_i(\varrho)$, for $i \in \{1, 2\}$.

Proposition 3 (conditional sample-path ordering). The result applies to a candidate wave W^k evaluated from the common initial state in Section 3.1, for any $E \geq 1$ and $c \geq 1$. Its conclusion holds when the two deterministic evaluations satisfy conditions (a)–(d).

- (a) Both evaluators process orders in the same recorded sequence π^k .
- (b) Each order is assigned to the same AMR in both evaluations.
- (c) No shared M_2 trip overtakes its corresponding M_1 requests. For every M_2 trip τ containing at least two requests, $\mathcal{G}_\tau$ denotes its request group and T_τ its shared unloading completion time. The required relation is

$$\max_{\varrho \in \mathcal{G}_\tau} D_1(\varrho) \leq T_\tau \quad (44)$$

- (d) Repositioning does not disadvantage M_1 on requests for which both evaluators create a new trip.

For each such request with origin floor g_ϱ , $G_{\text{sel}}^{(i)}(\varrho)$ denotes the selected resource's stored floor immediately before its state update in M_i . The required relation is

$$\left| G_{\text{sel}}^{(1)}(\varrho) - g_\varrho \right| \leq \left| G_{\text{sel}}^{(2)}(\varrho) - g_\varrho \right| \quad (45)$$

Under these conditions, every corresponding elevator request and the wave makespan satisfy

$$D_1(\varrho) \leq D_2(\varrho), \quad C_{\max}(W^k, \pi^k; M_1) \leq C_{\max}(W^k, \pi^k; M_2) \quad (46)$$

Equivalently, when (a) and (b) hold, a reversal of either inequality in Equation (46) requires (c) or (d) to fail.

Conditions (a) and (b) align the sequence, origins, and destinations of elevator requests, including the AMR's movement to an order's pickup floor. The recorded sequence in Equation (1), shared across evaluators, supplies (a). Condition (b) is an additional property of the paired evaluations because earlier elevator completions affect subsequent earliest-available-AMR choices. In (d), the stored floor is the resource's location at its recorded next-available time, as defined in Section 3.5. Conditions (c) and (d) are assessed on the paired reservation paths.

Proof sketch. The proof proceeds by induction over the reservation sequence. The induction maintains the completion-time ordering for previous requests and the ordering of the earliest E resource availabilities: if $x_{(1)} \leq \dots \leq x_{(Ec)}$ and $y_{(1)} \leq \dots \leq y_{(E)}$ are the sorted M_1 and M_2 availability states,

then $x_{(i)} \leq y_{(i)}$ for $i = 1, \dots, E$. Common AMR assignments and the monotone service-time recursions give an earlier or equal request time under M_1 . When both evaluators create a new trip, this comparison, earliest availability, and (d) order the new completion times. Replacing the smallest availability in each list preserves the required ordering. For a request joining an existing M_2 trip, condition (c) gives its completion-time comparison directly. The availability comparison follows from a counting argument. Any M_2 trip already replaced by a later reservation completed no later than the current fleet minimum; hence completions above a threshold $y_{\{(i)\}}$ can belong only to the latest trips

of at most $E - i$ cars. Those trips contain at most $(E - i)c$ requests. If the joining trip ends above that threshold, its spare place reduces the count of previous requests by one. Previous completion-time ordering transfers this count to the M_1 slots, preserving $x_{(i)} \leq y_{(i)}$ after the new slot reservation. The AMR recursions then order every order completion, including orders that inherit earlier resource delays without making a new elevator request. Taking the maximum and subtracting the common t_w proves Equation (46).

4.3.2 Reversal mechanisms and class-level implications

Two small instances isolate the roles of shared trips and repositioning. Both use $t_w = 0$, $f^0 = 1$, $E = 1$, $c = 2$, $\gamma = 5$, $\tau^L = \tau^U = 2$, and $\tau^P = \tau^D = 5$, with times measured in seconds. Both examples use the recorded processing sequence in the order described below. In the first instance, $F = 3$ and $A = 2$. Two orders travel from floor 1 to floor 3, with ready-time offsets 0 and 2. Their delivery requests reach the elevator at times 5 and 7. Under M_1 , separate slots complete the trips at 19 and 21, and the wave finishes at 26. Under M_2 , the second request joins the first trip at its loading-end time 7. Both requests finish the elevator trip at 19, giving a wave makespan of 24. Thus, a shared trip can overtake a corresponding independent-slot request, violating (c). In the second instance, $F = 8$ and $A = 1$. The first order travels from floor 1 to floor 8, and the second from floor 8 to floor 7; both have ready-time offset 0. The first order finishes at 49, and the second order's elevator request is issued at 54. Under M_1 , the earliest-available slot is the unused slot at floor 1. Its empty movement to floor 8 takes 35 seconds, producing a wave makespan of 103. Under M_2 , the car is already at floor 8, and the wave finishes at 68. No trip is shared; the reversal arises from the adverse repositioning distance in (d).

The examples show why (c) and (d) enter the sufficient conditions of Proposition 3: each example violates exactly one of them and reverses the ordering, so neither condition can be dropped. Conversely, a violated condition need not produce a reversal. The first example is a batch overtaking, in which a shared M_2 trip overtakes its corresponding M_1 requests; the second is an adverse repositioning. When AMR assignments differ, request alignment itself must also be examined.

Two registered checks [NS] concern whether these two mechanisms are exhaustive. An exhaustive enumeration of 1,000,800 instances with two or three orders, together with a seeded sample of 150,000 four-order instances (six floors, car capacity 2, cross-floor orders, one or two elevators, AMR counts and ready offsets on fixed grids; Appendix B), found no makespan reversal among instances that satisfy (a) to (d); this is a certification on that class, not a proof. On the 6,000 matched-wave draws of the preregistered ordering test (publication scale, counted with multiplicity; Section 5.2), replayed with common AMR assignments, each of the 20 reversals violates (c) or (d): 18 violate (c), and in the other 2, (c) holds and (d) fails.

The same replay shows that the sufficient conditions are restrictive. On these draws, (c) fails on 12.7% and (d) on 99.4%, so (a) to (d) hold jointly on fewer than 1% of them, whereas the makespan ordering holds on 99.7% of them with common assignments; with the free assignments of the preregistered test, the reported per-wave ordering rate is 99.2% (Section 5.2). The ordering therefore

holds far more often than the conditions certify, and Section 5 reports the frequency of the conditions and the frequency of the ordering separately.

The abstraction bias of candidate k is $B_k = [C_{\max}(W^k, \pi^k; M_2) - C_{\max}(W^k, \pi^k; M_1)] / C_{\max}(W^k, \pi^k; M_1)$, the relative amount by which the throughput abstraction understates the co-occupancy makespan; it is nonnegative whenever Equation (46) holds.

For a class q , the same draw $\kappa \sim \nu_q$ is evaluated under both deterministic evaluators. If conditions (a) to (d) of Proposition 3 hold for every candidate in $\mathcal{K}_q$, then $Y_{\kappa M_1} \leq Y_{\kappa M_2}$ almost surely. The class distribution function is $F_{q_i}(t) = \Pr_{\kappa \sim \nu_q}(Y_{\kappa M_i} \leq t)$. The almost-sure comparison implies first-order stochastic ordering and, consequently, midpoint-median ordering:

$$F_{q1}(t) \geq F_{q2}(t) \quad \text{for all } t, \quad \mu_{qM_1}^{0.5} \leq \mu_{qM_2}^{0.5}. \quad (47)$$

Fig. 4 contrasts the sufficient ordering conditions with the two reversal mechanisms.

(a) Conditional evaluator ordering · 4.3.1

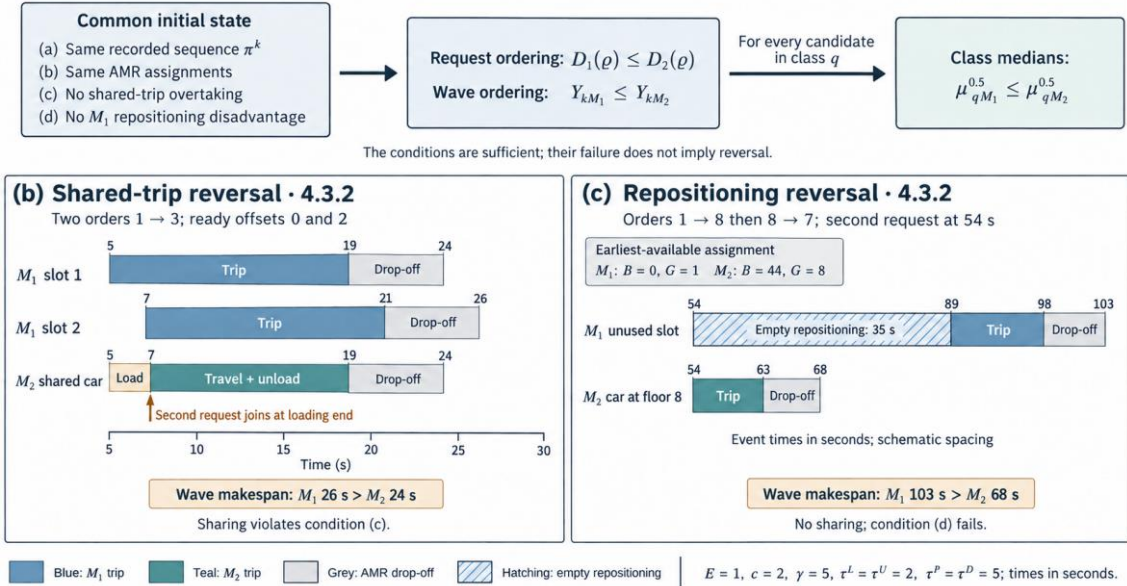


Fig. 4. Conditional ordering and two reversal mechanisms. (a) From a common initial state, the same recorded sequence and AMR assignments, together with no batch overtaking and no M_1 repositioning disadvantage, imply request and wave completion ordering. The conditions holding for every candidate in a class also imply class-median ordering. (b) Batch overtaking gives wave makespans of 26 s under M_1 and 24 s under M_2 . (c) Adverse repositioning gives 103 s and 68 s, respectively; event times are exact, while horizontal spacing is schematic. These examples violate different sufficient conditions; a violated condition alone does not guarantee reversal.

4.4 Properties of robust selection

4.4.1 Reduction under ordered evaluator scores

The procedure in Section 4.1 compares both deterministic class medians. The ordering analysis in Section 4.3 motivates examining when one evaluator alone determines the robust score. The relevant medians, robust score, and conservative choice are denoted by

$$a_q = \mu_{qM_1}^{0.5}, \quad b_q = \mu_{qM_2}^{0.5}, \quad V_q = \max\{a_q, b_q\}, \quad q_H \in \arg \min_{q \in Q} b_q. \quad (48)$$

Here b_q specializes the fixed-evaluator notation of Section 4.2 to M_2 . We refer to q_H as the M_2 -

based conservative choice. The same question arises for single candidates, whose scores ℓ_{km} , $m \in \mathcal{M}_D$, under the serial reservation (closed-form) evaluators of Section 3.5 are defined in Equation (16).

Corollary 1 (conditional minimax reduction). (a) If Equation (47) holds for every $q \in \mathcal{Q}$, or more generally if $a_q \leq b_q$ for every $q \in \mathcal{Q}$, then the robust class objective equals the M_2 median class by class, and

$$\arg \min_{q \in \mathcal{Q}} \max\{a_q, b_q\} = \arg \min_{q \in \mathcal{Q}} b_q. \quad (49)$$

(b) If a finite nonempty candidate set $\tilde{\mathcal{K}}$ satisfies $\ell_{kM_1} \leq \ell_{kM_2}$ for every $k \in \tilde{\mathcal{K}}$, then

$$\arg \min_{k \in \tilde{\mathcal{K}}} \max_{m \in \mathcal{M}_D} \ell_{km} = \arg \min_{k \in \tilde{\mathcal{K}}} \ell_{kM_2}. \quad (50)$$

Proof. (a) Equation (47) gives $a_q \leq b_q$. The ordering gives $V_q = b_q$ for every selectable class, so both minimizations have the same objective. (b) The per-candidate ordering gives $\max_{m \in \mathcal{M}_D} \ell_{km} =$

ℓ_{kM_2} for every $k \in \tilde{\mathcal{K}}$, and the same argument applies. $\square$ Proposition 3 supplies, through Equation (47), one sufficient route to the premise of (a) and, directly, one route to the premise of (b). Both premises can also be checked on the class medians or on the candidate scores, without requiring request-level ordering for every candidate. With a common tie convention, both minimizations return the same class or candidate.

Part (b) underlies the screening step of Section 4.5. If $\ell_{kM_1} \leq \ell_{kM_2}$ for every candidate of a screened pool, the M_2 score and the minimax score coincide on that pool, so they give the same ranking and, with a common tie order, the same shortlist of every size. Checking the premise requires the M_1 scores, and so do conditions (c) and (d) of Proposition 3, which are assessed on the paired reservation paths; screening by the M_2 score alone therefore rests on the ordering being assumed, for example from ordering rates measured on comparable candidates, except when $c = 1$, where the two evaluators coincide. The same algebraic reduction applies to the estimated objective in Section 4.1 whenever the estimated class medians are ordered. Request-level condition checks, makespan-ordering frequencies, and class-median comparisons are recorded separately, preserving the distinction between the premise of Proposition 3 and the premise of Corollary 1.

4.4.2 Departures from ordering and selection stability

When some class medians reverse order, their effect on the robust objective can be represented exactly by the positive excess of the M_1 median:

$$e_q = [a_q - b_q]_+, \quad [u]_+ = \max\{u, 0\}, \quad V_q = b_q + e_q. \quad (51)$$

Corollary 2 (excess loss and ranking stability). For any M_2 -optimal class q_H , its excess robust objective satisfies

$$0 \leq V_{q_H} - \min_{q \in \mathcal{Q}} V_q \leq e_{q_H}. \quad (52)$$

If q_H is the unique M_2 -optimal class, its ranking margin is $\Delta_H = \min_{q \neq q_H} (b_q - b_{q_H})$. This margin gives the sufficient condition

$$\Delta_H > e_{q_H} \implies \arg \min_{q \in \mathcal{Q}} V_q = \{q_H\}. \quad (53)$$

Proof. Since $V_q \geq b_q \geq b_{q_H}$, the optimal robust value is at least b_{q_H} . Subtracting this lower bound

from $V_{q_H} = b_{q_H} + e_{q_H}$ gives Equation (52). Under the strict margin condition, every other class satisfies $V_q \geq b_q > b_{q_H} + e_{q_H} = V_{q_H}$, proving Equation (53). $\square$

When the margin condition does not hold, the robust decision is obtained by comparing the full scores V_q . Corollary 2 assesses excess performance loss relative to the two-model median-minimax optimum. Evaluator-specific regret, such as the M_1 regret $a_{q_H} - \min_q a_q$, makes a different comparison and is reported separately. Corollary 2 does not involve M_3 , whose class distribution in Equation (14) is assessed on its own.

Remark 2. Under the ordering of Equation (47), the worst-case class mean over the 1-Wasserstein ball centered on the M_1 class distribution, with radius equal to the 1-Wasserstein distance between the M_1 and M_2 class distributions, equals the M_2 class mean, a mean-based counterpart of Corollary 1(a) and a simple instance of an analytical worst case under stochastic dominance (cf. Fu, Li and Zhang, 2024), whereas Equation (15) uses medians.

4.5 Generate, screen, and verify release procedure

The class-selection procedure of Section 4.1 releases a wave drawn from a class, and no such rule can release a wave whose score under an evaluator $m \in \mathcal{M}_D$ is below the pool optimum of Equation (17) for that evaluator. This section describes a release procedure that works with individual candidates instead. It generates candidate waves, screens them with a closed-form evaluator, and verifies a shortlist with an event-driven evaluator before release. The dispatch rule of Section 3 stays fixed; the procedure changes only the released wave, that is, its order set and its processing sequence.

The generate, screen, and verify (GSV) procedure has three steps (Algorithm 1). The generation step adds to the random pool candidates designed around the two mechanisms of Section 4.3. The screening step ranks every candidate with a closed-form evaluator, which is inexpensive enough to apply to thousands of candidates. The verification step evaluates the best-ranked candidates with an event-driven evaluator, in which AMRs act concurrently, and releases the best of them. The closed-form evaluators therefore serve as a filter, and the event-driven evaluator makes the final choice.

4.5.1 Candidate generation

The augmented pool is

$$\mathcal{K}^+ = \mathcal{K} \cup \mathcal{K}^{\text{seq}} \cup \mathcal{K}^{\text{con}}. \quad (54)$$

Its first part is the random pool $\mathcal{K}$ of Section 3.3.1 with the recorded sequences π^k . The second part, $\mathcal{K}^{\text{seq}}$, re-sequences each random candidate. The sequence-aware re-ordering keeps the order set W^k and changes only the sequence. It groups orders with the same source and destination floors into blocks of at most c orders, so that their elevator requests can share an M_2 trip. It then orders the blocks by readiness and, among equally ready blocks, chooses next the block whose source floor equals, or else is nearest to, the destination floor of the previous block, which is intended to reduce empty repositioning. The third part, $\mathcal{K}^{\text{con}}$, contains K' candidates from a co-ride-aware constructive generator. Starting from a random cross-floor order, the generator repeatedly adds the order with the highest weighted score. The score rewards completing a partly filled co-ride block and chaining to the wave (the order's source is the destination of an order already in the wave, or its destination is the source of one), and it penalizes endpoint floors new to the wave; the weights are fixed by a registered tuning rule on training pools, and ties are broken at random. Each constructed order set is then sequenced by the same re-ordering. For $k \in \mathcal{K}^{\text{seq}} \cup \mathcal{K}^{\text{con}}$, π^k denotes the sequence set by the re-ordering; every evaluator processes it in place of the recorded sequence of Equation (1), and the

statements of Section 3 about recorded sequences refer to the random pool $\mathcal{K}$. Both operators target the mechanisms of Section 4.3: under M_2 , requests with the same source and destination that are issued no later than the end of a trip’s loading, while the car has free capacity, share the trip, and linked floors can shorten repositioning. Appendix X specifies both operators, including their tie rules.

4.5.2 Closed-form screening

Each candidate $k \in \mathcal{K}^+$ is scored in closed form as in Equation (16), $\ell_{km} = C_{\max}(W^k, \pi^k; m)$ for $m \in \mathcal{M}_D$. The screening key is the M_2 score,

$$\psi_k = \ell_{kM_2}, \quad k \in \mathcal{K}^+. \quad (55)$$

This choice applies the robust objective of Section 3.4.2 to single candidates: if $\ell_{kM_1} \leq \ell_{kM_2}$ holds for every candidate of $\mathcal{K}^+$, the M_2 key and the minimax key $\max_{m \in \mathcal{M}_D} \ell_{km}$ coincide on $\mathcal{K}^+$ (Corollary 1(b)), so they give the same ranking and the same shortlist.

Proposition 3 gives sufficient conditions for the per-candidate ordering, and Section 4.3.2 identifies the two mechanisms that can reverse it. Re-sequenced and constructed candidates are built to create shared trips, so the ordering may fail more often on them. The procedure can therefore use the conservative key $\psi_k = \max_{m \in \mathcal{M}_D} \ell_{km}$ for $k \in \mathcal{K}^{\text{seq}} \cup \mathcal{K}^{\text{con}}$, keeping $\psi_k = \ell_{kM_2}$ for $k \in \mathcal{K}$; this key equals the minimax key on all of $\mathcal{K}^+$ when the ordering holds on $\mathcal{K}$. Section 5 states which key the registered study uses and the rule that decides it. Ties in ψ_k are broken by a fixed random order of the candidates. Candidates with the same sequence of source floors, destination floors, and ready offsets receive the same score from every evaluator, so they count once. The shortlist $\mathcal{K}^\vee$ contains the first $K_\vee$ distinct sequences in increasing order of ψ_k .

4.5.3 Event-driven verification and release

The closed-form evaluators of Section 3.5 process a candidate’s sequence π^k one order at a time, assigning each order to the earliest available AMR. The verification evaluator executes the same M_2 rules as an event-driven simulation. AMRs claim orders in the sequence π^k , act concurrently, wait for each order’s ready time, and request elevators in time order; a request boards an open trip with the same source and destination while its loading window is open and the car has free capacity, and otherwise takes the earliest available car. We denote this evaluator by M_2^{DES} and its makespan by $\ell_{kM_2^{\text{DES}}}$. The two evaluators can rank the same candidates differently; one reason is that the closed-form evaluators reserve elevator resources in the order of the sequence rather than in the time order of the requests. In the registered event-driven cross-check on publication-scale waves, rerun with the boarding rule used here, the closed-form ranking does not meet the registered criterion for a faithful ordinal proxy of the event-driven ranking (Section 5.5). The release decision is therefore made under M_2^{DES} . The procedure evaluates every shortlisted candidate and releases

$$k^\dagger \in \arg \min_{k \in \mathcal{K}^\vee} \ell_{kM_2^{\text{DES}}}, \quad (56)$$

with ties broken by the same fixed order. The screening regret

$$\text{Reg}(K_\vee) = (\ell_{k^\dagger M_2^{\text{DES}}} - \min_{k \in \mathcal{K}^+} \ell_{kM_2^{\text{DES}}}) / \min_{k \in \mathcal{K}^+} \ell_{kM_2^{\text{DES}}} \quad (57)$$

is the relative excess of the released candidate’s makespan over the best makespan in $\mathcal{K}^+$ under the verification evaluator. Because the shortlists are nested, $\text{Reg}(K_\vee)$ does not increase with $K_\vee$, and it is zero once $\mathcal{K}^\vee$ contains every distinct sequence. The smallest shortlist size whose regret falls within a stated tolerance, $K_\vee^*$, indicates how far closed-form screening can be trusted as a filter for the

verification evaluator; Section 5 estimates $\text{Reg}(K_V)$ and K_V^* .

Per released wave, the procedure uses at most $|\mathcal{K}^+|$ closed-form evaluations under M_2 (and under M_1 for candidates screened with the conservative key) and at most K_V event-driven evaluations. M_1 , M_2 , and M_2^{DES} are thus rungs of a fidelity ladder: the closed-form score filters candidates quickly, and M_2^{DES} makes the final choice. Section 4.3 gives conditions under which M_2 is the larger, and hence conservative, closed-form score, and Section 5 measures how often it is. The regret $\text{Reg}(K_V)$ measures how well the filter ranks candidates for the verification evaluator.

Algorithm 1. Generate, screen, and verify (GSV) release of one wave

Input: order pool $\mathcal{O}$; wave size n ; system parameters of Section 3.2 (including car capacity c); numbers of random and constructed candidates K and K' ; shortlist size K_V ; fixed tie order

Output: released wave (W, π)

- 1: Draw K random candidates (W^k, π^k) as in Section 3.3.1 *(generate)*
- 2: For each random candidate, form its sequence-aware re-ordering
- 3: Build K' candidates with the co-ride-aware constructive generator; sequence each by the same re-ordering
- 4: $\mathcal{K}^+ \leftarrow$ random, re-ordered, and constructed candidates
- 5: For every $k \in \mathcal{K}^+$, compute the screening key ψ_k (Equation (55)), or the conservative key for re-ordered and constructed candidates *(screen)*
- 6: Rank $\mathcal{K}^+$ by ψ_k , ties by the fixed order; $\mathcal{K}^V \leftarrow$ the first K_V distinct sequences
- 7: For every $k \in \mathcal{K}^V$, evaluate $\ell_{kM_2^{\text{DES}}}$ under M_2^{DES} *(verify)*
- 8: Release the $k \in \mathcal{K}^V$ with the smallest $\ell_{kM_2^{\text{DES}}}$, ties by the fixed order (Equation (56))

Algorithm 1 lists the steps for one wave. Lines 1 to 4 build the augmented pool of Equation (54). Line 2 leaves every order set unchanged, so the re-ordered candidates isolate the effect of the processing sequence, and line 3 adds order sets chosen for co-occupancy. Line 5 is the only evaluation step that touches every candidate, which is why it uses a closed-form evaluator. Line 6 counts distinct sequences, so the verification budget is not spent on duplicates, which constructed candidates in particular can produce. Lines 7 and 8 spend the event-driven evaluations only on the shortlist. The fixed tie order in lines 6 and 8 makes the released wave reproducible and prevents any candidate source from being favored by its position. When waves are released consecutively, the evaluations in lines 5 and 7 are made on the candidate appended to the waves already released, as one concatenated sequence evaluated from the common initial state; AMR and elevator states thus carry over, and Proposition 3 applies to the concatenation. Section 5 describes this warm-start use.

The procedure acts on the tactical release decision only. In the closed-form evaluators, the procedure and the clustered dispatch rule examined in Section 5.5 both act through the processing sequence; they differ in which layer sets it. The procedure sets the sequence at release, whereas clustered dispatch is an operational rule, equivalent in the evaluator to clustering the elevator's waiting queue, that serves here as an operational reference. Section 5 reports the closed-form effect of re-ordering at release next to that of clustered dispatch, each labeled with the scale at which it was measured, without ranking the two.

4.6 What Section 5 tests

Sections 4.1 to 4.5 contain two kinds of statements. Propositions 1 to 3, Corollaries 1 and 2, and Remarks 1 and 2 hold on any candidate pool that satisfies their premises; Section 5 does not test them but measures how often their premises hold on the benchmark and what happens when they do not. The class diagnostic, the choice of evaluator, and the release procedure make empirical claims, and Section 5 examines each against registered criteria.

Study 1 (Section 5.2) reports, with its verdicts unchanged, the preregistered tests of the class diagnostic and class rule of Sections 4.1 and 4.2 and of the evaluator ordering behind Sections 4.3 and 4.4: how often $\ell_{kM_1} \leq \ell_{kM_2}$ holds per wave and how often the class distributions are ordered as Corollary 1(a) requires. It also reports how often conditions (a) to (d) hold and, as a registered re-analysis, the abstraction bias B_k .

Study 2 (Section 5.3) evaluates the procedure of Section 4.5 against random, class-based, heuristic, and search policies. Under the event-driven evaluator, it asks how much the processing sequence alone changes makespan, whether closed-form screening of the augmented pool outperforms heuristic scoring at the same verification budget and how it compares with a budget-matched local search, how large a shortlist the verification step needs, and how much is lost by selecting with each closed-form evaluator. With the closed-form evaluators, it asks whether the ordering persists on candidates built to create shared trips. It also asks whether single-wave results carry over to consecutive waves, how robust the released waves are to execution noise, and how long each step takes. Section 5.4 applies the procedure to a literature-calibrated case, and Section 5.5 states where the closed-form diagnostics stop being reliable.